

Java数据结构 之



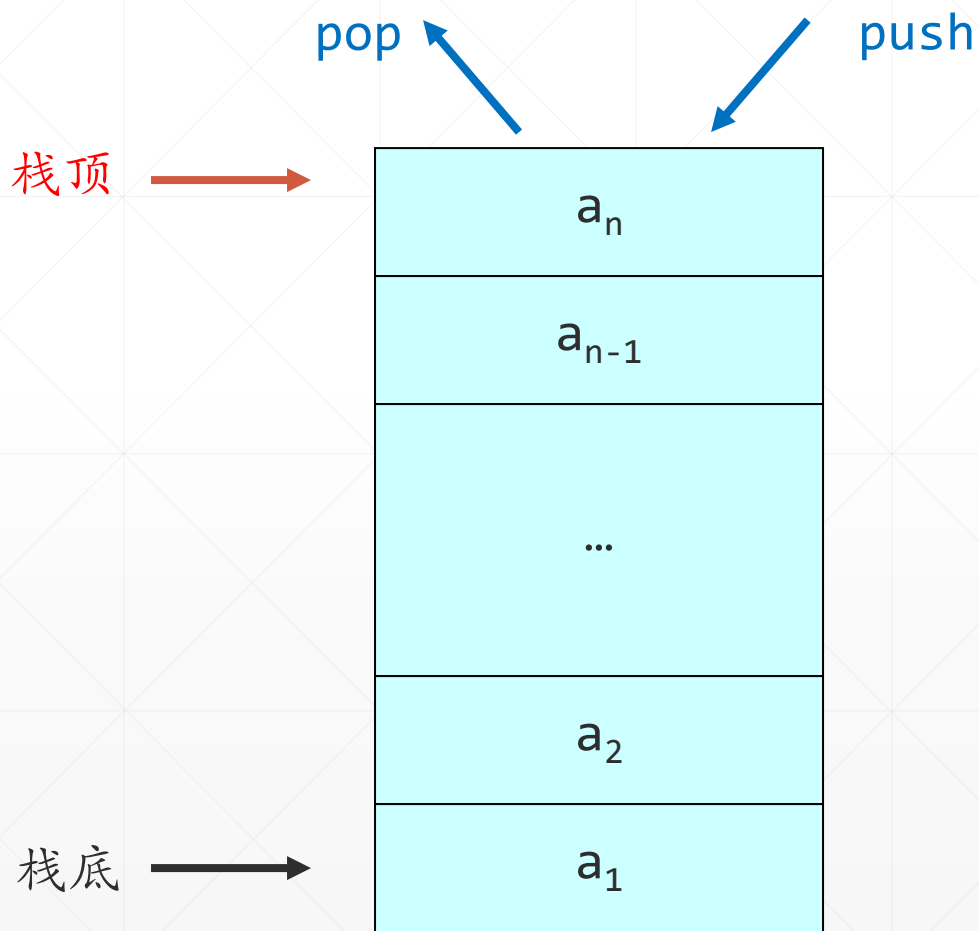
栈与队列

北京理工大学计算机学院
金旭亮

栈 (stack)



栈(Stack)是限制在一端进行插入和删除操作的线性表，具有后进先出（**LIFO: Last In First Out**）特性。



栈的基本操作

- 向栈中加入元素叫“**入栈 (push)**”
- 取出元素叫“**出栈 (pop)**”

用Java实现栈

在开发中如果需要自己实现一个栈，可以使用线性表（比如数组和链表等）作为内部存储机制，实现堆栈的基本操作。

本讲重用我们在前一讲中介绍过的“链表”类实现堆栈。

```
MyLinkedList
MyLinkedList(String)
MyLinkedList()
insertAtFront(T) void
insertAtBack(T) void
removeFromFront() T
removeFromBack() T
print() void
search(T) T
empty boolean
```

```
MyLinkedListNode
MyLinkedListNode(T)
MyLinkedListNode(T, MyLinkedListNode<T>)
next MyLinkedListNode<T>
data T
```

```
EmptyListException
EmptyListException(String)
```

有两种不同的方式实现堆栈：

使用继承方式
(StackInheritance.java)

使用组合方式
(StackComposition.java)

使用继承方式实现栈-1

//使用继承方式实现堆栈

```
public class StackInheritance
    extends MyLinkedList<String> {
    //构造方法
    public StackInheritance() {
        super(s: "stack");
    }
    //入栈
    public void push(String o) {
        insertAtFront(o);
    }
    //出栈
    public String pop()
        throws EmptyListException {
        return removeFromFront();
    }
}
```

StackInheritance.java

从MyLinkedList<T>中派生，入栈和出栈，实际上调用的是链表类中的相应方法。

//栈是否为空?

```
public boolean isEmpty() {
    return super.isEmpty();
}
```

//打印栈中的内容

```
public void print() {
    super.print();
}
```

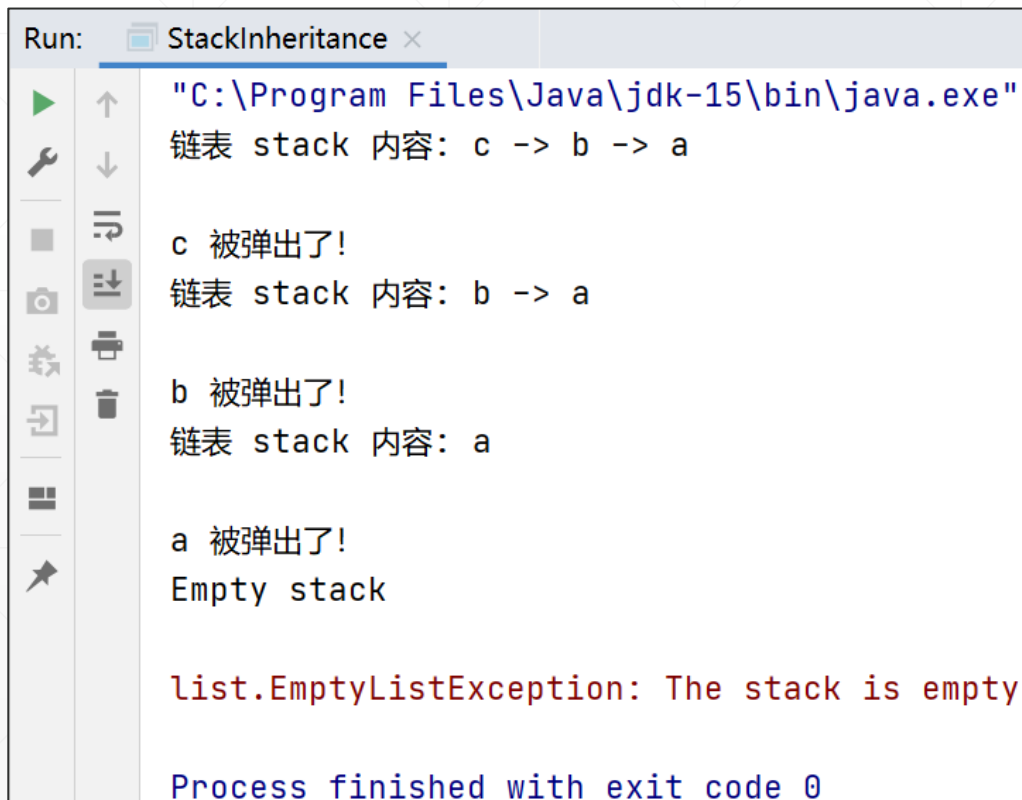
```
public static void main(String[] args) {...}
```

```
}
```


使用继承方式实现栈-2

```
//测试自定义堆栈的功能
public static void main(String[] args) {
    //创建一个堆栈对象
    var objStack = new StackInheritance();
    //向栈中压入三个字符串
    objStack.push("a");
    objStack.push("b");
    objStack.push("c");
    //输出栈的当前内容
    objStack.print();
    try {
        while (true) {
            //不断地出栈, 直到栈空, 抛出EmptyListException
            System.out.println(objStack.pop() + " 被弹出了!");
            //每弹出一个元素, 输出一次栈中剩余的元素
            objStack.print();
        }
    } catch (EmptyListException e) {
        System.err.println("\n" + e.toString());
    }
}
```

运行结果:



```
Run: StackInheritance x
"C:\Program Files\Java\jdk-15\bin\java.exe"
链表 stack 内容: c -> b -> a

c 被弹出了!
链表 stack 内容: b -> a

b 被弹出了!
链表 stack 内容: a

a 被弹出了!
Empty stack

list.EmptyListException: The stack is empty

Process finished with exit code 0
```

使用继承方式实现栈-3

//测试自定义堆栈的功能

```
public static void main(String[] args) {  
    //创建一个堆栈对象  
    var objStack = new StackInheritance();  
    objStack.|
```

//向栈中

objSta

objSta

objSta

//输出栈

objSta

```
try {
```

m push(String o)

m print()

m pop()

m isEmpty()

m insertAtFront(String insertItem)

m insertAtBack(String insertItem)

m removeFromBack()

m removeFromFront()

m search(String searchData)

虽然可以使用继承链表的方式实现堆栈的功能，但是使用者在使用它时，会发现有一堆与“堆栈”功能无关的方法可以调用（比如insertAtFront之类），会引发不必要的困惑。

另外，“继承”代表两个事物之间是“IS_A（是一种）”关系，但在这里，你实在不好说——堆栈是一个链表！



仅出于重用代码的目的而使用继承，是一种很糟的设计决策！

使用组合方式实现堆栈

//使用组合方式实现堆栈

StackComposition.java

```
public class StackComposition {  
    //在内部组合一个链表对象，并设置为私有的  
    private final MyLinkedList<String> linkList;  
  
    //在构造方法中实例化一个链表对象  
    public StackComposition() {  
        linkList = new MyLinkedList<String>(s: "stack");  
    }  
  
    "调用MyLinkedList相应的功能实现堆栈的功能"  
  
    public static void main(String[] args) {...}  
}
```

从“堆栈”实例中看到.....



继承与组合是实现代码复用的两种基本思路之一。

作为一个指导性的设计原则，我们有：

1. 能用组合实现的，就用组合，尽量减少使用继承。
2. 当且仅当两个事物之间存在“IS_A”关系时，才使用继承。

队列 (Queue)



队列(Queue)是一种操作受限的线性表。它只允许在表的一端进行插入，而在另一端进行删除。允许删除的一端称为**队头(front)**，允许插入的一端称为**队尾(rear)**。



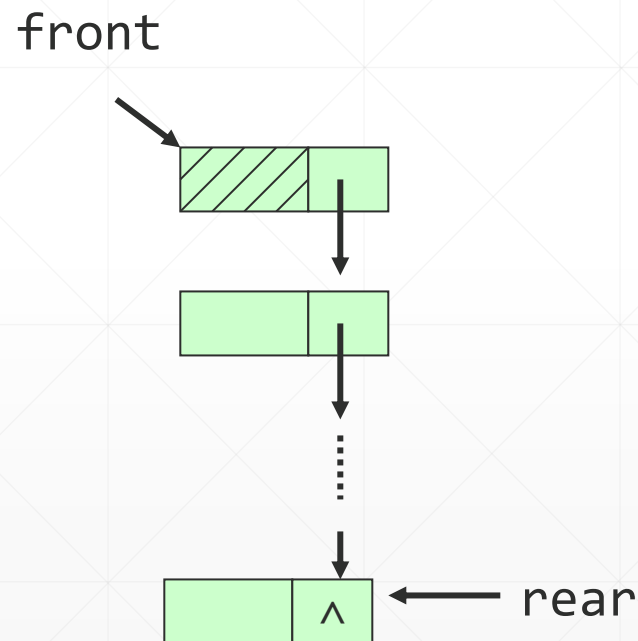
用Java实现队列

与堆栈类似，实现队列的最简单方式也是基于链表进行二次封装。

以下示例展示了使用继承实现队列的方式：

`QueueInheritance.java`

请先不看代码，先自己实现，之后再对照示例，然后，再用组合的方式实现队列（组合方式就不提供参考代码了）。



动手动脑



1. 编写一个程序，此程序读入一个英文句子，然后使用**栈**将其倒序输出。对比前面使用**双向链表**的类似编程练习，你认为这两种实现方式哪种好？
2. 使用堆栈检验括号的匹配

假设在表达式中

`( [ ] ( ) )` 或 `[ ( [ ] [ ] ) ]`

等为正确的格式，

`[ ( ] )` 或 `( [ ( ) )` 或 `( ( ) ] )`

均为不正确的格式。

编程完成检验一个表达式中括号是否匹配的任务。

动手动脑之项目实战

当前银行储蓄所普遍采用了取号排队的方式，请编写一个“仿真”程序，模拟这一“储户取号排队顺序办理银行业务”的现实生活场景。



提示：

- 这个仿真程序中包容一个等待队列和多个工作队列。
- 注意储户是随机到来的，而每个工作人员办理不同储户业务的时间有长有短。
- 想真正做到与真实系统高度一致，需要掌握Java多线程开发的知识，可以在以后学会多线程开发技术之后，再重做此题。
- 当前可以使用敲回车和敲空格键的方式进行模拟——敲空格表示又来了一个顾客，敲回车表示一个顾客办完了业务，离开了储蓄所。